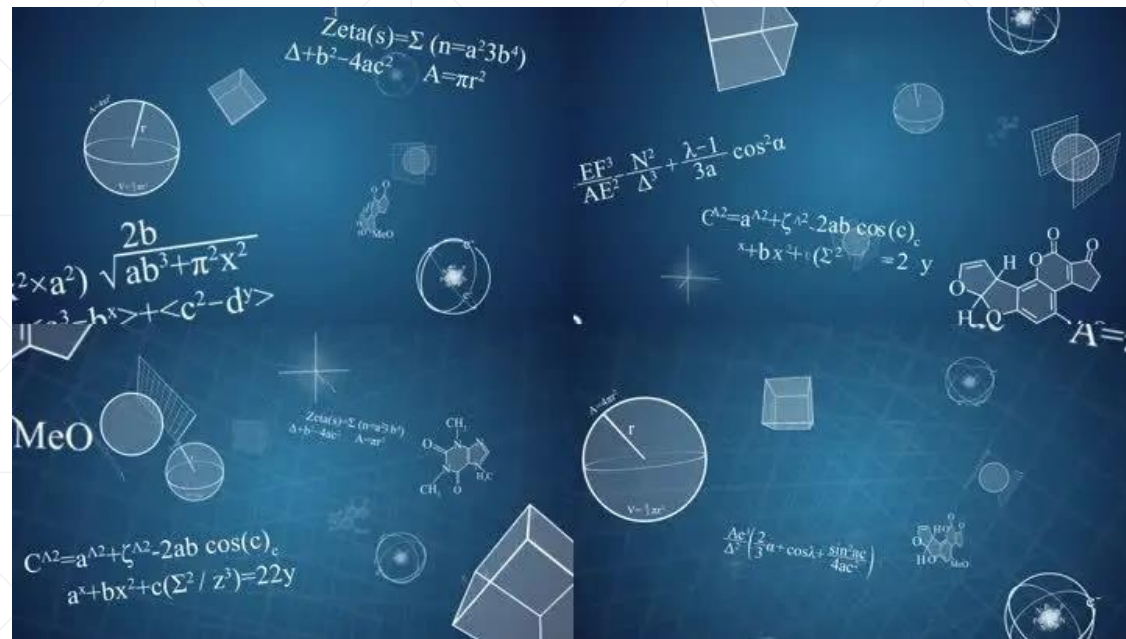




JavaFX专有多线程组件的使用

北京理工大学计算机学院
金旭亮

引子



前一讲我们介绍了JavaFX多线程开发的基础编程技巧，从本质上说，JavaFX是Java平台上的一个UI开发框架，因此，JDK中可用的多线程组件，往往都可以直接用于JavaFX应用。



JavaFX提供了一个Platform.runLater()方法，可以实现跨线程访问UI控件。知道了这点，已经没有什么能阻碍你开发JavaFX多线程应用了。



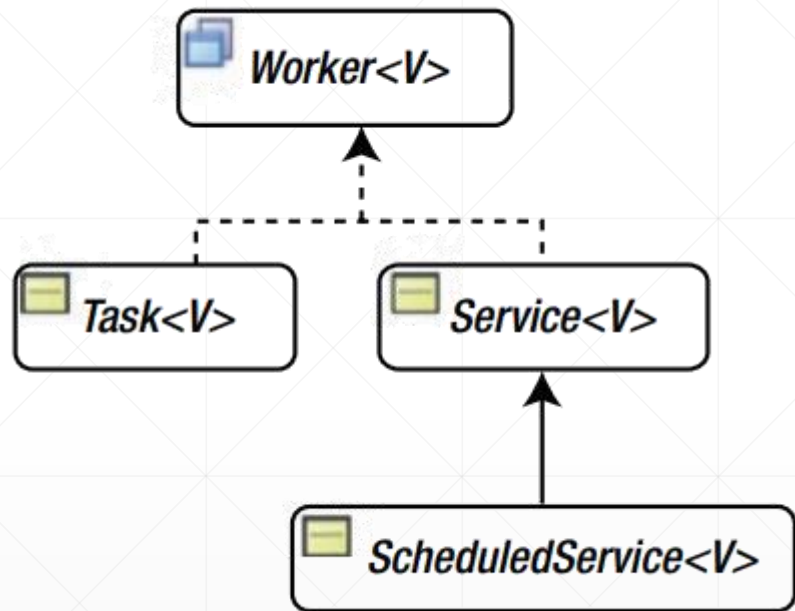
尽管如此，JavaFX还是提供了另外一些专属的多线程组件，解决了特定场景下的特定问题，在合适的地方使用它们，比“一切自己用原生多线程技术解决”，要高效得多，本讲就向大家介绍这部分组件。

一种非常常见的场景



用户通过UI控件启动某个计算任务，这个任务在后台运行，同时，需要在UI界面上显示其工作进度，需要时可以提前取消.....

JavaFX SDK中的多线程组件



JavaFX提供了一个`Worker<V>`接口，这是实现“后台运行计算，前台显示进度”的关键。

`Task<V>`抽象类实现了`Worker<V>`接口，它代表一个“仅运行一次”的多线程工作任务。

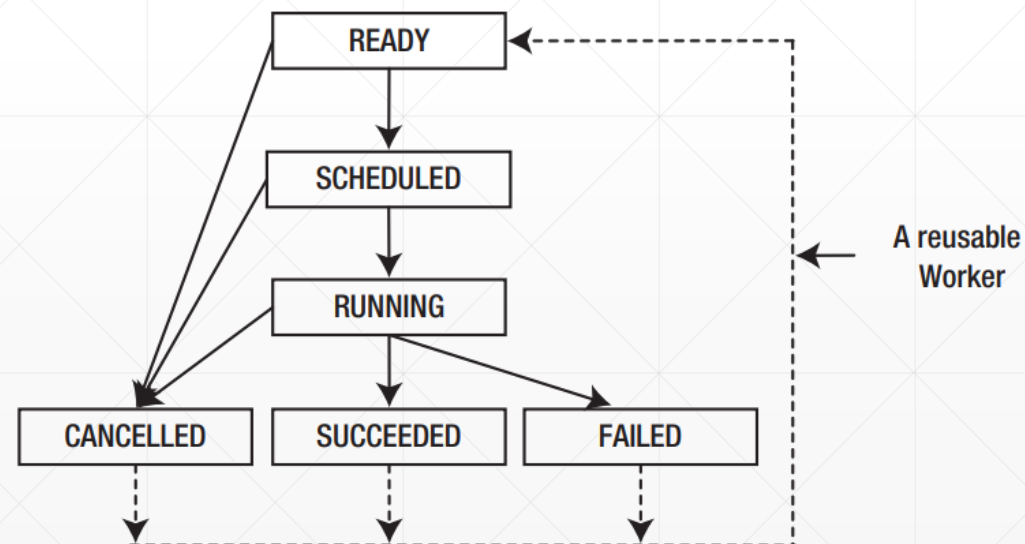
`Service<V>`则代表可以“重用”的多线程工作任务，它的子类`ScheduledService<V>`，代表一个可以定时调用的任务。

Worker<V>

- State
 - values() : State[]
 - valueOf(String) : State
 - State()
 - READY
 - SCHEDULED
 - RUNNING
 - SUCCEEDED
 - CANCELLED
 - FAILED
- getState() : State
- stateProperty() : ReadOnlyObjectProperty<State>
- getValue() : V
- valueProperty() : ReadOnlyObjectProperty<V>
- getException() : Throwable
- exceptionProperty() : ReadOnlyObjectProperty<Throwable>
- getWorkDone() : double
- workDoneProperty() : ReadOnlyDoubleProperty
- getTotalWork() : double
- totalWorkProperty() : ReadOnlyDoubleProperty
- getProgress() : double
- progressProperty() : ReadOnlyDoubleProperty
- isRunning() : boolean
- runningProperty() : ReadOnlyBooleanProperty
- getMessage() : String
- messageProperty() : ReadOnlyStringProperty
- getTitle() : String
- titleProperty() : ReadOnlyStringProperty
- cancel() : boolean

Worker类的实例，代表一个可以由一个或多个后台线程完成的工作任务。它的泛型参数V，代表工作任务的处理结果类型。

工作任务的状态（如下图所示）是一个枚举类型的JavaFX Bean Property，可以被“观察”，或者被其他的JavaFX控件所“绑定”。



JavaFX多线程组件示例



仅能使用一次的Task任务的执行结果



可以重用的Service任务的执行结果



定时调用任务的执行结果

执行前如果有其它任务在执行，会先取消它们的执行

创建一次性任务



泛型参数Integer
表示结果类型

更新进度

任务的创建

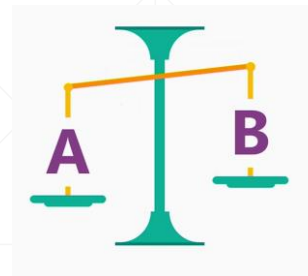
```
public class MyTask extends Task<Integer> {  
    @Override  
    protected Integer call() throws Exception {  
        for (int i = 0; i < 100; i++) {  
            Thread.sleep(200);  
            if (i % 5 == 0)  
                updateProgress(i, 100);  
        }  
        return 100;  
    }  
}
```

任务的定义

```
private MyTask createTask() {  
    MyTask task = new MyTask();  
    task.progressProperty().addListener(obj -> {  
        lblInfo.setText("已经完成" + (int) (task.getProgress() * 100) + "%");  
    });  
    task.valueProperty().addListener(obj -> {  
        lblInfo.setText("结果为: " + task.getValue());  
    });  
    return task;  
}
```

监听进度信息和最终结果，更新UI界面。

一次性任务的两种运行方式



//使用后台线程运行

1 usage

```
private static void runTaskUseThread(MyTask task) {  
    Thread thread = new Thread(task);  
    thread.setDaemon(true);  
    thread.start();  
}
```

//使用线程池运行

1 usage

```
private static void runTaskUseThreadPool(MyTask task) {  
    ExecutorService executor = Executors.newSingleThreadExecutor();  
    executor.submit(task);  
    executor.shutdown();  
}
```



不管是使用线程池还是直接创建自己的线程，都要考虑到用户有可能直接关闭程序退出，所以，要妥善处理好“后台工作未完成，而用户已关闭App”的问题。

可重用性任务



```
public class MyService extends Service<ObservableList<Long>> {  
    1 usage  
    @Override  
    protected Task<ObservableList<Long>> createTask() {  
        return new MyServiceTask();  
    }  
}
```

由于任务是可以反复执行的，所以，需要重写createTask()方法，以便确定需要执行的任务是哪个。

可重用的任务，同样需要派生自Task<T>基类，并重写其call()方法。

```
public class MyServiceTask extends Task<ObservableList<Long>> {  
    @Override  
    protected ObservableList<Long> call() throws Exception {  
        ObservableList nums = FXCollections.observableArrayList();  
        for (int i = 0; i < 100; i++) {  
            Thread.sleep(millis: 100);  
            if (i % 10 == 0) {  
                nums.add(i);  
                updateMessage("加入数值: " + i);  
            }  
        }  
        return nums;  
    }  
}
```

可重用性任务的运行



```
btnService.setOnAction(e -> {  
    cancelAllTask();  
    myService = new MyService();  
    myService.start();  
    myService.messageProperty().addListener(obj -> {  
        lblInfo.setText(myService.getMessage());  
    });  
    myService.valueProperty().addListener(obj -> {  
        lblInfo.setText("结果: " + myService.getValue().toString());  
    });  
});
```

Service类的start()方法，必须在UI线程中调用，它在内部会调用createTask()方法创建任务，然后，在后台运行.....

Service类所定义的消息和value属性，都是JavaFX Bean Property，所以它可以被监听，也可被绑定.....

定时任务的定义



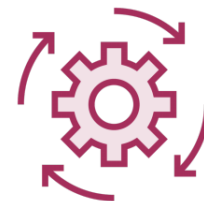
```
public class MyScheduleService extends ScheduledService<Integer> {  
    1 usage  
    @Override  
    protected Task<Integer> createTask() {  
        return new MyScheduledTask();  
    }  
}
```

实现定时任务的关键，在于选择这个基类

```
public class MyScheduledTask extends Task<Integer> {  
    @Override  
    protected Integer call() throws Exception {  
        var returnValue = ThreadLocalRandom.current().nextInt();  
        System.out.println(returnValue);  
        return returnValue;  
    }  
}
```

实现定时任务的数据处理功能，没什么特殊

运行定时任务



```
btnScheduleService.setOnAction(e -> {  
    cancelAllTask();  
    scheduleService = new MyScheduleService();  
    scheduleService.setDelay(Duration.seconds(s: 3));  
    scheduleService.setPeriod(Duration.seconds(s: 1));  
    scheduleService.start();  
    scheduleService.lastValueProperty().addListener(obj -> {  
        lblInfo.setText("定时任务: " + scheduleService.getLastValue());  
    });  
});
```

设定任务启动时机
和间隔时间

取消正在运行的任务

2 usages

```
private void cancelAllTask() {  
    if (scheduleService != null) {  
        scheduleService.cancel();  
        lblInfo.setText("上次的定时任务已取消");  
        scheduleService=null;  
    }  
    if (myService != null) {  
        myService.cancel();  
        lblInfo.setText("myService已经取消");  
        myService=null;  
    }  
}
```

需要时，可以调用cancel()
方法取消正在运行的后台任
务.....

小结



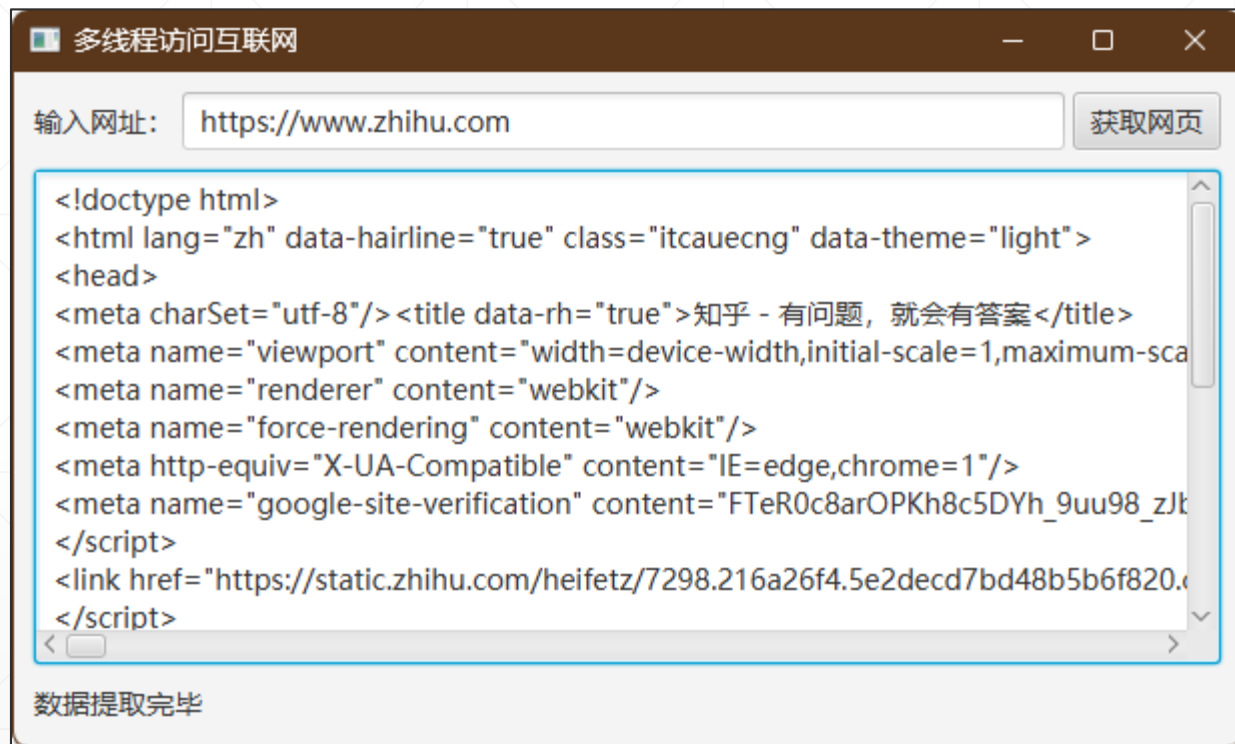
本讲通过几个示例，展示了JavaFX程序中，如何使用JavaFX多线程组件的基础编程技巧。



在实际开发中，以下场景，需要在独立的线程中完成工作：

- (1) 需要执行较长时间的运算
- (2) 从互联网上下载或上传数据
- (3) 存取本地硬盘文件
- (4) 从数据库中提取数据

巩固练习



请参照截图，使用学到的知识开发一个多线程访问Web网页的程序

此程序运行时，用户可以在文本框中输入一个网址，程序在后台线程中访问Web网页，提取数据后显示在TextArea中。

参考:MyWebApp.java